The algebra of the *random approximate map* model

Alexander Towell
atowell@siue.edu

Abstract

We present a formal algebraic framework for *random approximate maps*—probabilistic data structures that trade exactness for space efficiency. Building on rate-distortion theory, we develop a hierarchy of approximate map models characterized by false positive rate ε and false negative rate ω. We prove that compositions of first-order approximate maps yield higher-order approximations with quantifiable error bounds, and establish the relationship between approximate maps, sets, and relations through their indicator functions. For practical applications, we introduce the *singular hash map*, a theoretical construction that achieves optimal space complexity of $-\log_2 \varepsilon + \mu$ bits per element asymptotically. Our framework provides a unified foundation for analyzing probabilistic data structures including Bloom filters, sketch algorithms, and locality-sensitive hashing schemes.

Contents

1 Introduction **2**
1.1 Motivation and Context . . . 3
1.2 The Bernoulli Map Abstraction . . . 3
1.3 Contributions . . . 3
1.4 Paper Organization . . . 4

2 Related Work **4**
2.1 Probabilistic Data Structures . . . 4
2.2 Locality-Sensitive Hashing . . . 5
2.3 Information Theory and Rate-Distortion . . . 5
2.4 Approximate Computing . . . 5
2.5 Algebraic Approaches . . . 5
2.6 Encrypted Search and Privacy . . . 6
2.7 Distinction from Prior Work . . . 6

3 Approximate maps **7**
3.1 Rate-distortion . . . 9

4 Random approximate maps **9**
4.1 Approximate set-indicator function . . . 10
4.2 Probability model . . . 11

5 Higher-order random approximate maps **12**
5.1 Algebra of relations . . . 13
5.2 Algebra of sets . . . 13
5.3 Composition . . . 13

6 Confusion matrices for function spaces **14**
6.1 Function space confusion matrices . . . 14
6.2 Order and entropy . . . 15
6.3 Structural patterns in practice . . . 15
6.4 Examples of natural Bernoulli maps . . . 15
6.4.1 Set-indicator functions . . . 15
6.4.2 Primality testing . . . 16
6.4.3 Lossy compression . . . 16
6.5 Composition and higher-order effects . . . 16

7 Predicate functions **16**
7.1 Indicator functions (unary predicate functions) . . . 16
7.2 Relations as n-ary predicates . . . 16
7.2.1 Total orders . . . 17

8 Discrete partial functions **17**
8.1 0-ary functions (constants) . . . 17

9 A first-order rate-distorted map **17**
9.1 Maybe monad . . . 19
9.2 Theoretical analysis . . . 22
9.3 Entropy . . . 24

10 Conclusion **24**
10.1 Summary of Results . . . 24
10.2 Limitations and Open Questions . . . 24
10.3 Future Directions . . . 24
10.3.1 Theoretical Extensions . . . 25
10.3.2 Practical Applications . . . 25
10.3.3 Cross-Domain Impact . . . 25
10.4 Broader Impact . . . 25
10.5 Closing Remarks . . . 25

1 Introduction

Modern computing systems increasingly rely on probabilistic data structures to manage massive datasets within constrained memory budgets. These structures deliberately trade perfect accuracy for substantial space savings, enabling applications that would otherwise be computationally infeasible. Despite their widespread adoption in database systems, network monitoring, machine learning, and distributed computing, the theoretical foundations of probabilistic data structures remain fragmented across different domains.

In this paper, we introduce a unified algebraic framework for *random approximate maps*—a generalization that encompasses and extends existing probabilistic data structures. Our framework provides:

1. A rigorous mathematical foundation based on rate-distortion theory for analyzing approximation errors

2. A compositional algebra that predicts error propagation through complex operations

3. Optimal space-complexity bounds for various approximation scenarios

4. A hierarchy of approximation orders that captures increasingly complex error patterns

1.1 Motivation and Context

Consider the fundamental problem of representing a function $f : X \to Y$ in limited memory. A perfect representation requires $\log_2 |Y|^{|X|}$ bits in the worst case. However, if we accept a controlled probability of error, dramatic space reductions become possible. The Bloom filter [4], for instance, represents set membership (a special case where $Y = \{0, 1\}$) using only $-\log_2 \epsilon$ bits per element for false positive rate ϵ, independent of the universe size.

This space-accuracy tradeoff appears throughout computer science:

- **Streaming algorithms**: Count-Min sketch [11], HyperLogLog [17] approximate frequency and cardinality queries
- **Similarity search**: Locality-sensitive hashing [22] trades recall for speed in nearest-neighbor queries
- **Machine learning**: Random projections [23] preserve distances approximately while reducing dimensionality
- **Distributed systems**: Probabilistic data structures enable efficient synchronization and deduplication

Despite this diversity, these structures share fundamental algebraic properties that our framework captures systematically.

1.2 The Bernoulli Map Abstraction

We model approximate maps through the lens of *Bernoulli trials*. Given an objective function $f : X \to Y$, an approximate map $\tilde{f}$ exhibits two types of errors:

- **False positives** (rate ϵ): Elements outside f's domain appear to map to values
- **False negatives** (rate ω): Elements in f's domain fail to map correctly

The key insight is that these errors follow Bernoulli distributions when conditioned on membership in the domain. This probabilistic structure enables precise analysis of:

- Error propagation through function composition
- Algebraic operations on approximate sets and relations
- Optimal encoding strategies under rate-distortion constraints

1.3 Contributions

Our main contributions are:

1. **Formal Framework**: We develop a complete algebraic theory of approximate maps, including:
    - Precise definitions of approximation orders and their relationships

- Composition rules that predict error accumulation
- Connections to established concepts in information theory

2. **Theoretical Results**: We prove several fundamental theorems:
   - Lower bounds on space complexity for given error rates
   - Characterization of optimal encodings under various constraints
   - Conditions for preserving algebraic properties (associativity, distributivity) approximately
3. **Singular Hash Map**: We introduce a theoretical construction that:
   - Achieves information-theoretic optimal space complexity
   - Models first-order approximate maps exactly
   - Provides maximum entropy encoding for security applications
4. **Applications**: We demonstrate how our framework:
   - Unifies analysis of existing probabilistic data structures
   - Guides design of new structures with predictable error characteristics
   - Enables compositional reasoning about complex probabilistic systems

1.4 Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work on probabilistic data structures and approximation theory. Section 3 formally defines approximate maps and their rate-distortion characteristics. Section **??** develops the random approximate map model based on Bernoulli distributions. Section **??** analyzes compositions and higher-order approximations. Section **??** presents the singular hash map construction and proves its optimality. Section **??** demonstrates applications to existing data structures. Section **??** provides empirical validation of our theoretical predictions. Section 10 concludes with future research directions.

2 Related Work

Our work builds upon and unifies several research areas spanning theoretical computer science, information theory, and systems design.

2.1 Probabilistic Data Structures

The study of space-efficient probabilistic data structures began with Bloom filters [4], which represent sets with one-sided error. Numerous variants have been proposed to address specific limitations:

- **Counting Bloom filters** [16] support deletion at increased space cost
- **Cuckoo filters** [15] achieve better space efficiency and support deletion
- **Quotient filters** [2] provide cache-friendly implementations

For frequency estimation, the Count-Min sketch [11] and Count sketch [9] provide probabilistic guarantees on query accuracy. The AMS sketch [1] estimates second moments in data streams. These structures exemplify the space-accuracy tradeoffs our framework formalizes.

2.2 Locality-Sensitive Hashing

Indyk and Motwani [22] introduced locality-sensitive hashing (LSH) for approximate nearest neighbor search. Subsequent work developed LSH families for various distance metrics:

- Hamming distance [18]
- Jaccard similarity [6]
- Cosine similarity [10]

Our framework views LSH as approximate maps from points to hash buckets, where collision probabilities encode similarity. This perspective enables unified analysis of different LSH schemes.

2.3 Information Theory and Rate-Distortion

Shannon's rate-distortion theory [27] provides fundamental limits on lossy compression. Cover and Thomas [12] present comprehensive treatment of these concepts. Our work adapts rate-distortion principles to discrete functions rather than continuous signals.

The connection between data structures and information theory has been explored by:

- Mitzenmacher and Upfal [24] analyze Bloom filters using information-theoretic arguments
- Carter et al. [7] establish lower bounds for approximate membership queries
- Pagh et al. [26] prove optimality of certain hashing schemes

2.4 Approximate Computing

The broader field of approximate computing trades accuracy for efficiency across the computing stack:

- **Hardware**: Approximate arithmetic circuits [20]
- **Algorithms**: Randomized approximation algorithms [25]
- **Systems**: Approximate query processing [21]

Our framework provides theoretical foundations specifically for data structure approximations.

2.5 Algebraic Approaches

Algebraic methods in computer science have been studied extensively:

- **Abstract data types**: Goguen et al. [19] formalized ADTs algebraically
- **Monoids and sketches**: The monoid structure of sketches enables parallel and distributed computation [5]
- **Category theory**: Categorical approaches to data structures provide compositional semantics [3]

Our contribution extends these algebraic approaches to probabilistic settings, defining operations that preserve approximation guarantees.

2.6 Encrypted Search and Privacy

Approximate data structures play crucial roles in privacy-preserving systems:

- **Private set intersection**: Bloom filters enable efficient PSI protocols [13]
- **Searchable encryption**: Approximate structures hide access patterns [8]
- **Differential privacy**: Sketches provide privacy through approximation [14]

Our framework's emphasis on oblivious computation aligns with these privacy requirements.

2.7 Distinction from Prior Work

While previous work studied specific probabilistic data structures or general approximation concepts, our contribution is a unified algebraic framework that:

1. Formalizes the relationship between maps, sets, and relations through approximation
2. Provides compositional semantics for complex approximate operations
3. Establishes hierarchy of approximation orders with precise error characterization
4. Connects practical data structures to fundamental information-theoretic limits

This systematic approach enables both theoretical analysis and practical design of novel probabilistic data structures. This association may only be partial over the domain, in which case it is a *partial function* and otherwise is a *total function*. A total function is commonly called just a *function*. A partial function $\mathrm{f}\colon \mathcal{X} \rightharpoonup \mathcal{Y}$ may be thought of as a total function $\mathrm{f}\colon \mathcal{X} \mapsto \bot\mathcal{Y}$.

Let $\mathrm{f}\colon \mathcal{X} \mapsto \mathcal{Y}$ be a function. The concept of a Bernoulli map of f, denoted by $\mathrm{AF}(f)\colon \mathcal{X} \mapsto \mathcal{Y}$, models f with *error*, either do to rate distortion, noise, or ignorance, such that

$$\mathrm{P}\big[\mathrm{AF}(f)(x) \neq \mathrm{f}(x) \,\big|\, x \in \mathcal{A}\big] = \epsilon_{\mathcal{A}}\,. \tag{1}$$

with two general categories of error:

1. Elements in the domain of definition may be undefined.
2. Undefined elements may be defined.

There are also random approximate value types.

The void and unit types have respectively 0 and 1 elements, therefore there are no approximations of these types. The most primitive type which has an approximation is the approximate Boolean type, in which which under the *random approximate type model*, the element $\mathbf{true}^{\tau}_{\varepsilon}$ is Bernoulli distributed that realizes **true** with probability τ and **false** with probability ε and similarly for $\mathbf{false}^{\tau}_{\varepsilon}$.

Given void, unit, and Bool^{σ}, we may algebraically compose them to generate other types. For instance, the product type $\mathsf{Bool}^{\sigma} \times \mathsf{Bool}^{\sigma}$ is a random approximate product of Booleans. By the product rule in probability,

$$\mathrm{P}\Big[\big\langle \mathbf{true}^{\tau_1}_{\varepsilon_1}, \mathbf{true}^{\tau_2}_{\varepsilon_2}\big\rangle = \langle \mathbf{true}, \mathbf{true}\rangle\Big] = \tau_1\tau_2\,. \tag{2}$$

Under the random approximate Boolean model, the logical functions $\wedge\colon \mathsf{Bool} \times \mathsf{Bool} \mapsto \mathsf{Bool}$ and $\neg\colon \mathsf{Bool} \mapsto \mathsf{Bool}$ are also random. In particular, $\neg(\mathbf{true}^{a}_{b})$ and $\neg(\mathbf{false}^{a}_{b})$ are respectively distributed as $\mathbf{false}^{b}_{a}$ and $\mathbf{true}^{b}_{a}$.

These results generate the random approximate Boolean algebra

$$\left(\mathsf{Bool}_{\varepsilon_1}^{\tau_1} \times \mathsf{Bool}_{\varepsilon_2}^{\tau_2}, \wedge, \vee, \neg, \langle \mathbf{true}_{\varepsilon_1}^{\tau_1}, \mathbf{true}_{\varepsilon_2}^{\tau_2} \rangle, \langle \mathbf{false}_{\varepsilon_1}^{\tau_1}, \mathbf{false}_{\varepsilon_2}^{\tau_2} \rangle\right) , \tag{3}$$

which only *approximately* obeys the axioms.

Any set $\mathcal{U}$ with operations set-intersection and set-complement may be used to form the Boolean algebra

$$(\mathcal{P}(\mathcal{U}), \cap, \cup, \bar{\cdot}, \mathcal{U}, \varnothing) . \tag{4}$$

If $|\mathcal{P}(\mathcal{U})| = 4$, then it is *isomorphic* to the Boolean algebra given by eq. (3). The *approximate algebra of sets* over a universe $\mathcal{U}$ is thus given by

$$\left((\mathsf{Bool}_{\varepsilon}^{\tau})^n, \wedge, \vee, \neg, \left(\mathbf{true}_0^1\right)^n, \left(\mathbf{false}_0^1\right)^n\right) . \tag{5}$$

Functions of the type $\mathcal{X} \mapsto \mathcal{Y}$ where $\mathcal{X}$ and $\mathcal{Y}$ may be any algebraic type, e.g., $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2$ for a *binary function*, has an algebraic notation $\mathcal{Y}^{\mathcal{X}}$.

A *random approximate exponential type* is a partial or total function, $\widetilde{\mathrm{f}} \colon \mathcal{X} \rightharpoonup \mathcal{Y}$, with a domain of definition $\mathrm{def}(\widetilde{\mathrm{f}}_\tau^\varepsilon) = (\mathrm{def}(\mathrm{f}))_\tau^\varepsilon$ but whose range depends on the function. For example, if f is a bijection when restricted to its domain of definition, then $\mathrm{range}\left(\widetilde{\mathrm{f}}_\tau^\varepsilon\right) = (\mathrm{range}(\mathrm{f}))_\tau^\varepsilon$.

The approximate map is an example of a class of *approximate relations* where the relation has a functional constraint. Other constraints include properties like *symmetric* or *reflexivity*. Some constraints, however, are difficult to implement under the random approximate type model. However, note that, more generally, the approximation error can be applied to *any* predicate.

The approximate map is closely related to the *approximate set*[29] and *approximate relation*[28]. Approximate maps are one of the key components in an algebraic data type corresponding to the *exponential type*.

Bernoulli distributed, such that anything that composes them, e.g., a *random approximate set* over a universe of n elements may be modeled as the product of n random approximate Booleans.

In section 3, we precisely define the approximate map. In the subsequent sections, we explore properties of approximate maps, such as its *mean average precision* with respect to particular scoring functions. In section 5.3, we consider compositions of approximate maps, which are themselves approximate maps. In section 7, we show how the approximate map may optimally implement the abstract data type of the *approximate relation*. Finally, in later sections, we explore approximate rank-ordered search based on *first-order* random approximate maps.

3 Approximate maps

The concept of a *map* is dependent upon the concept of a *set*.

Definition 3.1. A set is an unordered collection of distinct elements from a universe of elements.

A countable set is a *finite set* or a *countably infinite set*. A *finite set* has a finite number of elements. For example,

$$\mathcal{S} = \{1, 3, 5\}$$

is a finite set with three elements. A *countably infinite set* can be put in one-to-one correspondence with the set of natural numbers.

The cardinality of a set $\mathcal{S}$ is a measure of the number of elements in the set, denoted by

$$|\mathcal{S}| . \tag{6}$$

The cardinality of a *finite set* is a non-negative integer and counts the number of elements in the set, e.g.,

$$\left|\{1,3,5\}\right| = 3\,.$$

Definition 3.2 (Cartesian product)**.** Let $\mathcal{A}$ and $\mathcal{B}$ be sets. The set $\mathcal{A}\times\mathcal{B} = \{(a,b)\colon a\in\mathcal{A}\wedge b\in\mathcal{B}\}$ is called the Cartesian product of the sets $\mathcal{A}$ and $\mathcal{B}$.

Definition 3.3 (Binary relation)**.** Let $\mathcal{A}$ and $\mathcal{B}$ be sets. By a relation $\mathcal{R}$ on $\mathcal{A}$ and $\mathcal{B}$, we mean a subset of the Cartesian product $\mathcal{A}\times\mathcal{B}$. Since the Cartesian product is over two sets, we denote this relation a *binary* relation.

Relations may be generalized to n-ary relations, which are subsets of the Cartesian product over n sets. We consider only *binary relations* known as *maps* as given by the following definition.

Definition 3.4. A *map* $\mathrm{f}\colon \mathcal{X}\mapsto\mathcal{Y}$ is a binary relation on $\mathcal{X}\times\mathcal{Y}$ with the constraint that the first element in the set of ordered pairs is *unique*.

Such a map is also known as a *partial function*.

Consider discrete partial maps of type $\mathcal{X}\rightharpoonup\mathcal{Y}$. Any function f of this type may be *lifted* to the type $\mathcal{X}\mapsto\mathcal{Y}_\perp$ where elements not in $\mathrm{def}(\mathrm{f})$ map to a special value denoted by nothing that is not a member of $\mathcal{Y}$.

Given a map, by convention, in an ordered pair (x,y), the first element x is denoted a *key* and the second element y is denoted the *value* associated with that (unique) key. The value y associated with key x may be denoted by

$$\mathrm{f}(x)\,. \tag{7}$$

The set of keys in the map is denoted by $\mathrm{dom}(\mathrm{f})\subseteq\mathcal{X}$ and the set of values is denoted by $\mathrm{codom}(\mathrm{f})\subseteq\mathcal{Y}$. Given these definitions, a map f is given by the set of ordered key-value pairs

$$\mathrm{f}\colon \mathcal{X}\mapsto\mathcal{Y} = \Big\{\big(x,\mathrm{f}(x)\big)\in\mathcal{X}\times\mathcal{Y}\colon x\in\mathrm{dom}(\mathrm{f})\Big\}\,. \tag{8}$$

A map $\mathrm{f}\colon \mathcal{X}\mapsto\mathcal{Y}$ may represent *many*-to-*one* relationships, where many keys may be related to the same value. Consequently, denoting a particular key by its associated value is not generally possible.

A *table* is a convenient way to represent a map, where each key $x\in\mathrm{dom}(\mathrm{f})$ is associated with $\mathrm{f}(x)\in\mathrm{codom}(\mathrm{f})$. The column for the keys consists of *unique* values from $\mathcal{X}$ and column for the values consists of (possibly repeated) values from $\mathcal{Y}$.

Example 3.5. Suppose we have a map $\mathrm{f}\colon\{1,3,7\}\mapsto\{2,7\}$ defined by the set of ordered key-value pairs

$$\{(3,2),(1,5),(7,2)\}\,. \tag{9}$$

We see that $\mathrm{dom}(\mathrm{f}) = \{3,1,7\}$, $\mathrm{codom}(\mathrm{f}) = \{2,5\}$, $\mathrm{f}(3) = 2$, $\mathrm{f}(1) = 4$, and $\mathrm{f}(7) = 2$. Notice that both 3 and 7 map to 2. Elements may be repeated in the 2nd index of ordered pairs, but elements in the 1st index must be unique.

Table 1 also depicts f where the *keys* column must consist of unique elements from $\mathcal{Z}$ and the *values* column must consist of (possibly repeated) elements from $\mathcal{Z}$.

Table 1: A table representing map $\mathrm{f}\colon \mathcal{Z} \mapsto \mathcal{Z}$.

domain	range
3	2
1	5
7	2

3.1 Rate-distortion

Suppose we have the discrete metric space $(\mathcal{Y}_\perp, \mathrm{d})$ where $\mathrm{d}\colon \mathcal{Y}_\perp \times \mathcal{Y}_\perp \mapsto \mathbb{R}_{\geq 0}$. If we have a distribution X with a sample space $\mathcal{X}$ then the *expected* distance with respect to f and g is given by

$$\mathrm{E}\big[\mathrm{d}(\mathrm{f},\mathrm{g})\big][\mathrm{X}] = \sum_{x \in \mathcal{X}} \mathrm{p}_\mathrm{X}(x)\, \mathrm{d}(\mathrm{f}(x), \mathrm{g}(x))\,. \tag{10}$$

If p_X is uniformly distributed, then $\mathrm{E}\big[\mathrm{d}(\mathrm{f},\mathrm{g})\big][\mathrm{X}]$ is the *average* distance $\bar{\mathrm{d}}(\mathrm{f},\mathrm{g})$ over all values.[1]

Suppose we have an objective map $\mathrm{f}\colon \mathcal{X} \mapsto \mathcal{Y}_\perp$ and an approximation of f denoted by $\mathrm{f}^\pm\colon \mathcal{X} \mapsto \mathcal{Y}_\perp$. Then, we call d on the metric space $(\mathcal{Y}_\perp, \mathrm{d})$ a *loss* function where the expected loss of the approximate function g of the objective function f is given by

$$\ell_\mathrm{X}(\mathrm{g} \mid \mathrm{f}) = \mathrm{E}\big[\mathrm{d}(\mathrm{f},\mathrm{g})\big][\mathrm{X}]\,. \tag{11}$$

By the axioms of probability, if it is given that the random variable $\mathrm{X} \in \mathscr{A}$ where $\mathscr{A}$ is a subset of sample space $\mathcal{X}$, then the conditional loss, denoted by $\ell_{\mathrm{X}|\mathscr{A}}$, is given by

$$\ell_{\mathrm{X}|\mathscr{A}}(\mathrm{g} \mid \mathrm{f}) = \mathrm{E}\big[\mathrm{d}(\mathrm{f},\mathrm{g})\big][\mathrm{X} \mid \mathscr{A}] \tag{12}$$

where $\mathrm{p}_{\mathrm{X}|\mathscr{A}}$ is normalized over $\mathscr{A}$.

Suppose the domain X may be partitioned into n disjoint sets $\mathrm{X}_1, \ldots, \mathrm{X}_n$. The expected loss may be rewritten as

$$\ell_{\mathrm{X}|\mathscr{K}}(g \mid f) = \sum_{i=1}^{n} \ell_{\mathrm{X}|\mathscr{K} \cap \mathrm{X}_i}(g \mid f)\,. \tag{13}$$

Ideally, an approximation g of f is selected or chosen such that $\ell_{\mathrm{X}|\mathscr{A}}(\mathrm{g} \mid \mathrm{f})$ is minimized over some subset $\mathscr{A} \subseteq \mathrm{X}$ of interest. However, the expected loss may not obtain 0 due to *rate-distortion*, incomplete knowledge, noise, and memory or computing constraints.

4 Random approximate maps

The set of functions of type $\mathcal{X} \mapsto \mathcal{Y}$ has cardinality $|\mathcal{Y}|^{|\mathcal{X}|}$. We may also denote functions of this type by $\mathcal{Y}^\mathcal{X}$. Given a function $\mathrm{f}\colon \mathcal{X} \mapsto \mathcal{Y}$, a randomly chosen function $\mathrm{g}\colon \mathcal{X} \mapsto \mathcal{Y}$ equals f when applied to $x \in \mathcal{X}$ with probability $\frac{1}{|\mathcal{Y}|}$ and therefore $\mathrm{g} = \mathrm{f}$ with probability $|\mathcal{Y}|^{-|\mathcal{X}|}$.

We generalize this result and consider *random approximate maps*.

Definition 4.1. The Iverson bracket, denoted by $[\![\,]\!]\colon \{\textbf{true}, \textbf{false}\} \mapsto \{0, 1\}$, maps a predicate to 1 if true and otherwise 0.

[1] One way of defining a related discrete metric space $(\mathcal{Y}_\perp^\mathcal{X}, \mathrm{d})$ where $\mathrm{d}\colon (\mathcal{X} \mapsto \mathcal{Y}_\perp) \times (\mathcal{X} \mapsto \mathcal{Y}_\perp) \mapsto \mathbb{R}_{\geq 0}$ is given by letting $\mathrm{d}(\mathrm{f},\mathrm{g}) \coloneqq |\mathcal{X}|\, \bar{\mathrm{d}}(\mathrm{f},\mathrm{g})$.

For example, $[\![a < b]\!] = 1$ if $a < b$ is **true**.

Let $\mathrm{Y}_x \coloneqq [\![\mathrm{f}(x) = \tilde{\mathrm{f}}(x)]\!]$ be a Boolean random variable. The probability that for all x in $\mathcal{X}$, $\mathrm{Y}_x = 1$ is given by

$$\mathrm{P}[\cap_{x\in\mathcal{X}} \mathrm{Y}_x = 1]\,. \tag{14}$$

If we assume independence,

$$\mathrm{P}[\cap_{x\in\mathcal{X}} \mathrm{Y}_x = 1] = \prod_{x\in\mathcal{X}} \mathrm{P}[\mathrm{Y}_x = 1]\,. \tag{15}$$

Since Y_x for each $x \in \mathcal{X}$ denote independent Boolean random variables, they are *Bernoulli* distributed.

Definition 4.2. TODO: talk about independence. The *order* of a random approximate map corresponds to the minimum number of partitions in which every element of a given partition is identically and independently Bernoulli distributed random variables. We model $\mathrm{f}\colon \mathcal{X} \mapsto \mathcal{Y}_\perp$ by a *random* approximate function denoted by $\mathrm{f}^{\pm}$. That is, $\mathrm{f}^{\pm}$ is a random function.

We denote a k-th order random approximate map of f by $\mathrm{f}^{\pm}_k$ or, if we know and wish to emphasize the partitions, $\mathrm{f}^{\pm}_{\mathcal{X}_1,\ldots,\mathcal{X}_k}$ or the Bernoulli probabilities corresponding to each partition, $\mathrm{f}^{\pm}_{p_1,\ldots,p_k}$.

Since each partition consists of some number of Bernoulli distributed random variables, that means each partition has a *binomially distributed* number of 1's.

In the random approximate map model, we assume that X is uniformly distributed.

4.1 Approximate set-indicator function

We may view a set $\mathcal{A}$ with respect to its *indicator* function $\mathbf{1}_{\mathcal{A}}\colon \mathcal{X} \mapsto \{0, 1\}$.

The set indicator function partitions $\mathcal{X}$ into two parts, $\mathrm{P} \coloneqq \{x \in \mathcal{X} : \mathbf{1}_{\mathcal{A}}(x) = 1\}$ and $\mathrm{N} \coloneqq \{x \in \mathcal{X} : \mathbf{1}_{\mathcal{A}}(x) = 0\}$.

Definition 4.3. The *false positive rate*, denoted by $\hat{\varepsilon}$, is defined as the expectation $\ell_{\mathrm{X|N}}(\mathrm{g} \mid \mathrm{f})$ and the *true negative rate*, denoted by $\hat{\eta}$, is defined as the probability $1 - \hat{\varepsilon}$.

Definition 4.4. The *false negative rate*, denoted by $\hat{\omega}$, is defined as the expectation $\ell_{\mathrm{X|P}}(\mathrm{g} \mid \mathrm{f})$ and the *true positive rate*, denoted by $\hat{\tau}$, is defined as the probability $1 - \hat{\tau}$.

Remark 4.5. If P is a priori known, we can ignore elements not in the domain of definition of f and thus discard $\hat{\varepsilon}$.

By eq. (13) and definitions 4.3 and 4.4, the expected loss over the entire domain is given by

$$\ell_{\mathrm{X}}(\mathrm{g} \mid \mathrm{f}) = \ell_{\mathrm{X|N}}(\mathrm{g} \mid \mathrm{f})\,\mathrm{p}_{\mathrm{X}}(\mathrm{N}) + \ell_{\mathrm{X|P}}(\mathrm{g} \mid \mathrm{f})\,\mathrm{p}_{\mathrm{X}}(\mathrm{P}) = \hat{\varepsilon}(1 - \mathrm{p}_{\mathrm{X}}(\mathrm{P})) + \hat{\omega}\,\mathrm{p}_{\mathrm{X}}(\mathrm{P})\,. \tag{16}$$

The false negative rate is given by $\omega = \ell_{\mathrm{X|P}}(\mathrm{f}, \mathrm{f}^{\pm})$.

The false positive rate is given by $\varepsilon = \ell_{\mathrm{X|N}}(\mathrm{f}, \mathrm{f}^{\pm})$.

Each element in N is a false positive with probability ε. That is, it is a Bernoulli trial with $p = \varepsilon$. If a false positive occurs, then it randomly chooses a *representation* in the prefix-free codes for $\mathcal{Y}$ or an undefined code, which we map to nothing.

If the values in $\mathcal{Y}$ are uniquely represented, then it randomly (uniformly) chooses a value in $\mathcal{Y}$.

It may hash to an undefined code. Two possibilities to consider:

1. The hash to undefined code that is not a prefix of any other code.
2. If the codes are variable-length, it may hash to a prefix of one or more codes.

If the values in $\mathcal{Y}$ are uniquely represented, then it randomly (uniformly) chooses a value in $\mathcal{Y}$.

4.2 Probability model

TODO 3:

General first-order random approximate map model is given by the following definition.

Definition 4.6. A function $\mathrm{f}_{\mathcal{A}^+}\colon \mathcal{X} \mapsto \mathcal{Y}$, $\mathcal{A} \subseteq \mathcal{X}$, is a *first-order* random approximate map of $\mathrm{f}\colon \mathcal{X} \mapsto \mathcal{Y}$ if the following conditions are satisfied:

$$\mathrm{f}_{\mathcal{A}^\pm}(x) \coloneqq \begin{cases} \mathrm{Y}^+ & x \notin \mathcal{A}\,, \\ \mathrm{Y}_- & x \in \mathcal{A}\,. \end{cases} \tag{17}$$

where Y^+ is a random variable with a support that is a subset of $\mathcal{Y}$ (including being equal to $\mathcal{Y}$).

If we have a first-order rate-distorted map, then Y^+ has a probability mass function that is given by

$$\mathrm{p}_{\mathrm{Y}^+}(y) = \sum_{b \in \mathtt{encode}_{\mathcal{Y}}(y)} 2^{-\ell(b)}\,, \tag{18}$$

where $\mathtt{encode}_{\mathcal{Y}}$ is a prefix-free encoder for values in $\mathcal{Y}$.

An optimal encoder may be found by computing the frequency of each element in $\mathcal{Y}$ when we apply f to every element in $\mathcal{A}$ or $\mathcal{X}$[2] and generating a Huffman code for it.

However, since encoding the Huffman code may require a relatively large amount of memory, especially if the image of f is large, this approach may not be practical. Alternatively, there are many well-known algorithms to generate prefix codes for specific distributions, or families of distributions. Universal codes may be appropriate when the image of f is very large. If all else fails, there is always the uniform distribution.

Example 4.7. The first-order positive random approximate set $\mathcal{A}^+_\varepsilon$ is a first-order random approximate map of $\mathbf{1}_{\mathcal{A}}\colon \mathcal{X} \mapsto \{0,1\}$ where, conditioning on $\mathcal{A}$, $\sum_{b \in \mathtt{encode}_{\{0,1\}}(1)} 2^{-\ell(b)} = \varepsilon$ and $\sum_{b \in \mathtt{encode}_{\{0,1\}}(0)} 2^{-\ell(b)} = 1 - \varepsilon$.

If there are constraints, such as the space of bijective functions $\mathcal{X} \mapsto \mathcal{X}$, then there are $|\mathcal{X}|!$ such functions and the probability that, given a bijective function f, a randomly chosen bijective function g is equal to f with probability $\frac{1}{|\mathcal{X}|!}$.

TODO -1:

The distance function $\mathrm{d}(a,a) = 0$ and otherwise 1 combined with ℓ counts the proportion of values mapped correctly.

TODO 0:

The *zero-th* order approximation? Maybe we say that its uniformly distributed, i.e., just a random variable with a sample space $\mathcal{Y}$.

What is first order then? Partition into k subsets, and elements in partition j are identically distributed.

What is second order? Partition into k subsets, and when we condition on a subset ...

TODO .5: Is 1st order actually 4th order model? Partions $\mathcal{X}$ into $\mathcal{A}$ and $\overline{\mathcal{A}}$. Then, it partitions $\overline{\mathcal{A}}$ into $\{y^*\}$ and $\bar{\mathcal{A}} - \{y^*\}$ and it partitions $\mathcal{A}$ given $x \in \mathcal{A}$ into $\{\mathrm{f}(x)\}$ and $\mathcal{Y} - \{\mathrm{f}(x)\}$. That makes 4 partitions, each element in which has a different conditional probability.

We shouldn't overcomplicate though. Maybe just say these are random approximate maps, sometimes difficult to quantify distributions, but in some cases we can provide a well-understood

[2]The latter may be chosen if we care about the approximation error over $\mathcal{X} \setminus \mathcal{A}$.

model, e.g., the set-indicator function has a first-order model that's straight-forward. Even higher-order models are somewhat understandable. Boolean algebra types in general, including $\{0, 1\}$, all have this easily understood first-order model?

TODO 1:

Revised first-order model: $\mathrm{f}_{A^{\pm}} \colon \mathcal{X} \mapsto \mathcal{Y}$ has the property that if $x \in \mathcal{A}$ then $\mathrm{Y} = \mathrm{f}_{\mathcal{A}^{\pm}}(x)$ realizes $\mathrm{f}(x)$ with probability $1 - r$ and is uniformly distributed over sample space $\mathcal{Y} - \{\mathrm{f}(x)\}$ otherwise, i.e., realizes any value in $\mathcal{Y} - \{\mathrm{f}(x)\}$ with probability $r/(|\mathcal{Y}| - 1)$.

For each $x \notin \mathcal{A}$, $\mathrm{Y} = \mathrm{f}_{\mathcal{A}^{\pm}}(x)$ is independently and identically distributed where it realizes some default value y^* with probability $1 - \varepsilon$ and is uniformly distributed over $\mathcal{Y} - \{y^*\}$ otherwise.

TODO 1.5:

Don't necessarily mention rate distortion except in passing, and then talk about, say, SHM as generating errors due to rate distortion.

TODO 2:

Given a function $\mathrm{f} \colon X \mapsto Y$, $\mathrm{f}^{q}_{Q^{\omega}_{\varepsilon}} \colon X \mapsto Y_q$ approximately models $\mathrm{f} \restriction_Q \colon X \mapsto Y$ where elements not in $Q \subseteq X$ are mapped by $\mathrm{f} \restriction_{Q^{\pm}}$ to Y with probability ε and to some *default* value q with probability $1 - \varepsilon$.

The *first-order random approximate set*, $\mathcal{A}^{\omega}_{\varepsilon}$, may be modeled by the approximate map of the characteristic function, $\mathrm{f}^{0}_{\mathcal{A}^{\omega}_{\varepsilon}} \colon \mathcal{U} \mapsto \{0, 1\}$.

If we wish to speak of the *type*, we may move the notation to the arrow, i.e., $X \overset{\varepsilon}{\underset{\omega}{\mapsto}} Y$.

The default value may be disjoint from Y, e.g., the sum type $Y + q$.

If we do not know the rates, or they are unimportant, we may instead denote it by $\mathrm{f}_{Q^{\pm}} \colon X \mapsto Y$.

$\mathrm{f} \colon X \mapsto Y$, $\mathrm{f}_{Q^{\omega}_{\varepsilon}} \colon X \mapsto Y$

We say that is of type $\mathrm{f} \colon X \mapsto Y$, $\mathrm{f}_{Q^{\omega}_{\varepsilon}} \colon X \mapsto Y$

Additionally, elements in Q are not mapped by $\mathrm{f}_{\restriction_{Q^{\pm}}}$.

TODO: revisit false negative and false positive rate. False positive rate should probably just be about what the approximate map is conditioned on, i.e., if f : X -> Y and we build amap(f,A) then we have fA : A -> m(Y). If fA(x) != Nothing when x not in A, then that is a false positive. Simplify this result. Now we can focus more on false negatives. A false negative is when fA(x) != f(x) when x in A. In other words, we want to look at the restriction of the original function f to A -> m(Y) (partial) and compare it to fA.

Now, for the approx boolean, fA : bool -> bool (or m(bool)), we typically let A = 0,1. So, in that case, there are no "negatives" and so no false positives, there is only a rate distortion on the positives. Only two positives, 0 and 1. If fA(false) maps to Nothing, that's a false negative... but shouldn't really happen in this case the function is so simple. Now we must also talk about the mabye monad though... because a approx(bool) is 0,1,nothing with some prob. distribution over the elements, P[fA(true) = nothing] is false negative rate. P[fA(true) != nothing] is 1-fnr. Rate distortion on positives that don't map to Nothing are a different matter. Now we're talking about

5 Higher-order random approximate maps

Typically, a data structure models a *first-order* random approximate map where the approximation error is due to *rate-distortion*. Then, through algebraic operations on first-order maps, such as composition, *higher-order* approximate maps are the natural result.

We discuss some of these higher-order models in this section.

5.1 Algebra of relations

5.2 Algebra of sets

The *set-complement*, *set-intersection*, and *set-union* operators have set-indicator functions defined respectively as

$$\mathbf{1}_{\overline{\mathcal{A}}} \coloneqq 1 - \mathbf{1}_{\mathcal{A}}(x)\,, \tag{19}$$

$$\mathbf{1}_{\mathcal{A}\cap\mathcal{B}}(x) \coloneqq \mathbf{1}_{\mathcal{A}}(x)\mathbf{1}_{\mathcal{B}}(x)\,, \tag{20}$$

$$\mathbf{1}_{\mathcal{A}\cup\mathcal{B}}(x) \coloneqq \mathbf{1}_{\mathcal{A}}(x) + \mathbf{1}_{\mathcal{B}}(x) - \mathbf{1}_{\mathcal{A}}(x)\mathbf{1}_{\mathcal{B}}(x)\,. \tag{21}$$

5.3 Composition

Composition of first-order approximate maps yields (potentially) a higher-order approximate map where elements in the negative set are not identically distributed.

The higher the order of the approximation, the greater the number of different distributions.

The positive set is more difficult to quantify since an algorithm may try to generate maps that yield minimal error.

If the ...

Compose should return a function that is the composition of a list of functions of arbitrary length. Each function is called on the return value of the function that follows. You can think of compose as moving right to left through its arguments.

Let $\widetilde{\mathrm{f}}^{\omega_1}_{\varepsilon_1} : \mathcal{X} \mapsto \mathcal{Y}$ be an approximate map with a false positive rate ε_1 and false negative rate ω_1 and let $\widetilde{\mathrm{g}}^{\omega_2}_{\varepsilon_2} : \mathcal{Y} \mapsto \mathcal{Z}$ be an approximate map with a false positive rate ε_2 and false negative rate ω_2.

Then, the composition $\widetilde{\mathrm{g}} \circ \widetilde{\mathrm{f}} : \mathcal{X} \mapsto \mathcal{Z}$ is an approximate map.

Assumption 5.1. When composing $\widetilde{\mathrm{f}}$ with $\widetilde{\mathrm{g}}$, if $x \in \mathrm{dom(f)}$, then $\mathrm{f}(x) \in \mathrm{dom(g)}$. Otherwise, the composition is undefined.

Let X denote a randomly chosen element from $\mathcal{X}$. Under the assumptions of our model, what is the probability that X results in undefined behavior when used as the input in the composition $\widetilde{\mathrm{f}}^{\omega}_{\varepsilon} \circ \widetilde{\mathrm{g}}^{\omega}_{\varepsilon}$?

$\overset{+}{-}\mapsto$

$\overset{\varepsilon}{-}\rightharpoonup$

$\overset{\omega}{\varepsilon}\mapsto$

$\overset{\varepsilon_3}{}$

$\overset{\omega_2}{\varepsilon_1}\mathcal{A}\overset{\omega_2}{\varepsilon_2}$

$\overset{\omega_3}{}$

$\overset{\varepsilon}{\mathcal{A}}$

$\overset{\omega}{\omega}\mathcal{A} \times \overset{\omega}{\varepsilon}\mathcal{A}$

$\overset{+}{-}\mathcal{A}$

$\overset{\pm}{}\mathcal{A}$

Suppose $\omega_1 = \omega_2 = 0$. If it is given that $\mathrm{X} \in \mathrm{dom(f)}$, the probability of a false positive is zero. Otherwise, a false positive occurs if:

1. $\mathrm{X} \in \mathrm{dom}(\widetilde{\mathrm{f}}^{\omega}_{\varepsilon})$ and $\widetilde{\mathrm{f}}^{\omega}_{\varepsilon}(\mathrm{X})$ maps to a value $\mathrm{Y} \in \mathrm{dom}(\widetilde{\mathrm{g}}^{\omega}_{\varepsilon})$.

Suppose we have an approximate map where the keys are from the tuple of n elements given by the type

$$\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_n \tag{22}$$

and the values are from the tuple of m elements given by the type

$$\mathcal{Y} = \mathcal{Y}_1 \times \cdots \times \mathcal{Y}_m \,. \tag{23}$$

Example 5.2. Let the keys be of the type

$$\mathcal{X} = \{1, 2\} \times \{a, b\} \tag{24}$$

and the values be of the type

$$\mathcal{Y} = \{c, d\} \times \{1, 2\} \,. \tag{25}$$

Since these are small sets, for may enumerate the universe of keys and values. The universe of keys is given by

$$(1, a), (1, b), (2, a), (2, b) \tag{26}$$

and the universe of values is given by

$$(c, 1), (c, 2), (d, 1), (d, 2) \,. \tag{27}$$

Then, the map $\mathrm{f} \colon \mathcal{X} \mapsto \mathcal{Y}$ maps tuples of the form $(x_1, \ldots, x_n), x_j \in \mathcal{X}_j$ to values of $(y_1, \ldots, y_m), y_j \in \mathcal{Y}_j$.

Currying is a way of constructing functions that allows partial application of a function's arguments.

Example 5.3. Let the approximate map $\mathrm{f} \colon \mathcal{Z} \times \mathcal{Z} \mapsto \mathcal{Q} \times \mathcal{Z}$ be defined by the rule

$$\mathrm{f}(x_1, x_2) = \left(\frac{x_2}{2}, x_1 \right) \tag{28}$$

for $(x_1, x_2) \in \{(1, 1), (2, 2), (3, 4)\}$. The set of ordered pairs defining f is thus given by

$$\left\{ \left((1, 1), \left(\frac{1}{2}, 1 \right) \right), ((2, 2), (1, 2)), \left((3, 4), \left(\frac{3}{2}, 4 \right) \right) \right\} . \tag{29}$$

Define composition of functions. Then, talk about how a composition of functions with approximate domains result in composition of functions with approximate domains. Talk about how the false positive rate increases. Relate this to an error measure?

One kind of composition is fuzzy hedge functions?

6 Confusion matrices for function spaces

The Bernoulli map framework quantifies approximation errors through confusion matrices over function spaces, revealing fundamental limits on function recovery and inference.

6.1 Function space confusion matrices

For functions $\mathrm{f} : \mathcal{X} \to \mathcal{Y}$, the complete confusion matrix has dimension $|\mathcal{Y}|^{|\mathcal{X}|} \times |\mathcal{Y}|^{|\mathcal{X}|}$.

Definition 6.1 (Function Confusion Matrix)**.** The confusion matrix $\mathbf{Q}$ for Bernoulli map approximations has entries:

$$Q_{ij} = \mathrm{P}\Big[\text{observe } \mathrm{p}_j \mid \text{latent } \mathrm{p}_i\Big] \tag{30}$$

where $\mathrm{p}_i, \mathrm{p}_j : \mathcal{X} \to \mathcal{Y}$ are functions indexed by $i, j \in \{1, \ldots, |\mathcal{Y}|^{|\mathcal{X}|}\}$.

Example 6.2 (Boolean Functions)**.** For $\mathrm{f} : \mathsf{Bool} \to \mathsf{Bool}$, there are exactly 4 functions:

- p_1: constant false ($\mathrm{p}_1(x) = \textbf{false}$ for all x)
- p_2: identity ($\mathrm{p}_2(x) = x$)
- p_3: negation ($\mathrm{p}_3(x) = \neg x$)
- p_4: constant true ($\mathrm{p}_4(x) = \textbf{true}$ for all x)

The 4×4 confusion matrix fully characterizes all possible approximations.

6.2 Order and entropy

The order of a Bernoulli map corresponds to the degrees of freedom in its confusion matrix.

Definition 6.3 (Bernoulli Map Order)**.** A Bernoulli map has order k if its confusion matrix has k independent parameters. The maximum order is $n(n-1)$ where $n = |\mathcal{Y}|^{|\mathcal{X}|}$.

Theorem 6.4 (Entropy Characterization)**.** *The entropy of a Bernoulli map approximation is:*

$$H(\mathit{Bernoulli}[\mathcal{X} \to \mathcal{Y}]) = -\sum_{i,j} Q_{ij} \log Q_{ij} \tag{31}$$

with conditional entropy given the latent function p_i*:*

$$H(\mathit{Bernoulli}[\mathcal{X} \to \mathcal{Y}] \mid \mathrm{p}_i) = -\sum_{j} Q_{ij} \log Q_{ij} \tag{32}$$

6.3 Structural patterns in practice

Real-world Bernoulli maps exhibit specific confusion matrix structures:

Theorem 6.5 (Diagonal Dominance)**.** *For most practical approximations with error rate* $\epsilon < 0.5$*:*

$$Q_{ii} \geq 1 - |\mathcal{X}| \cdot \epsilon \tag{33}$$

creating diagonal dominance that preserves function identity with high probability.

Theorem 6.6 (Sparsity from Independence)**.** *When errors are independent across domain elements:*

$$Q_{ij} = \prod_{x \in \mathcal{X}} \mathrm{P}\Big[\mathrm{AF}(p)_i(x) = \mathrm{p}_j(x)\Big] \tag{34}$$

Most off-diagonal entries become exponentially small in $|\mathcal{X}|$*.*

6.4 Examples of natural Bernoulli maps

6.4.1 Set-indicator functions

The indicator function $\mathbb{1}_A : \mathcal{X} \to \mathsf{Bool}$ becomes a Bernoulli map through structures like Bloom filters:

$$\mathrm{P}\big[\mathrm{Bernoulli}[\mathbb{1}_A](x) = \textbf{true} \mid x \notin A\big] = \varepsilon \tag{35}$$

The confusion matrix has special structure: transitions preserve subset relationships.

6.4.2 Primality testing

The Miller-Rabin test creates a Bernoulli approximation of is_prime : $\mathbb{N} \to \mathsf{Bool}$ with:

- False positive rate: $\leq 1/4^k$ for k rounds
- False negative rate: 0 (always correct for primes)
- Confusion matrix: highly asymmetric with zero entries

6.4.3 Lossy compression

A lossy codec $\mathrm{c} : \mathcal{X} \to \mathcal{X}$ has confusion matrix determined by rate-distortion:

$$Q_{ij} = \mathrm{P}\Big[\mathrm{c}(\mathrm{p}_i) = \mathrm{p}_j\Big] = \exp(-\lambda \cdot d(\mathrm{p}_i, \mathrm{p}_j)) \tag{36}$$

where d is a distortion metric and λ controls the rate-distortion tradeoff.

6.5 Composition and higher-order effects

Theorem 6.7 (Composition Order Increase)**.** *If* $\mathrm{AF}(f) : \mathcal{X} \to \mathcal{Y}$ *has order* k_1 *and* $\mathrm{AF}(g) : \mathcal{Y} \to \mathcal{Z}$ *has order* k_2*, then* $\mathrm{AF}(g) \circ \mathrm{AF}(f)$ *has order at most* $k_1 \cdot k_2 \cdot |\mathcal{Y}|$*.*

Proof. The composition's confusion matrix is the product of individual confusion matrices, with intermediate marginalization over $\mathcal{Y}$. □

These structural insights demonstrate that Bernoulli maps provide a unified framework for understanding diverse approximation schemes through their confusion matrix properties.

7 Predicate functions

7.1 Indicator functions (unary predicate functions)

The indicator function of a set $\mathcal{A}$ over a universal set $\mathcal{U}$ is denoted by $\mathbf{1}_{\mathcal{A}} \colon \mathcal{X} \mapsto \{0, 1\}$. A *first-order* random approximate set of $\mathcal{A}$ with a false positive rate ε and false negative rate ω is denoted by $\mathcal{A}^{\omega}_{\varepsilon}$, which may be modeled by a *first-order* random approximate map $\widetilde{\mathbf{1}^{\omega}_{\mathcal{A}_{\varepsilon}}} \colon \mathcal{X} \mapsto \{0, 1\}$. More generally, functions of type $\mathcal{X} \overset{\varepsilon}{\underset{\omega}{\mapsto}} \{0, 1\}$ model first-order random approximate subsets of the universal set $\mathcal{X}$. This may also be indicated by $\mathrm{R}^{\omega}_{\varepsilon}$ given that R may realize any element in $\mathcal{P}(\mathcal{X})$.

7.2 Relations as n-ary predicates

A k-ary predicate function is a k-ary *relation*. For the k-ary predicate on k oblivious types, we may use either *approximate relations* or perfect relations. However, a perfect relation for oblivious types requires a bit length that is a function of the total possible number of k-relations on the types whereas the approximate relation on oblivious types only requires a bit length that is a function of the the number of relations in the exact relation. Except for very small types, the approximate relation is the only practical option.

TODO: mention how the oblivious relation can be also oblivious to the *arity* of the relation. Could be a unitary relation, could be a k-ary relation where k is arbitrary large. This is good for preventing random "searches" over tuples from effectively determining actual relations.

TODO: need to flesh out the oblivious relation (approximate relation) paper. Then, a k-ary predicate is a k-ary relation. It's not necessarily a function, it's only asking if a given k tuples are *related*. For instance, an ordered set answers questions like $a \leq b$ if (a, b) is in the binary relation.

A k-ary predicate satisfying the *regular function* concept, $\mathrm{f} \colon T_1 \times \cdots \times T_k \mapsto \mathsf{Bool}$, maps k-tuples of type $(T_1, \ldots, T_k)$ to **true** or **false**.

Higher-order may result from many sources.

Suppose we have two binary predicates $\mathrm{f} \colon \mathcal{X} \times \mathcal{Y} \mapsto \{0, 1\}$ and $\mathrm{g} \colon \mathcal{Y} \times \mathcal{Z} \mapsto \{0, 1\}$. To model a k-ary predicate, we might chain the tests, e.g., a 3-ary predicate is approximated by $\mathrm{w} \colon \mathcal{X} \times \mathcal{Y} \times \mathcal{Z} \mapsto \{0, 1\}$ with a definition

$$\mathrm{w}(x, y, z) \coloneqq \mathrm{f}(x, y) \wedge \mathrm{g}(y, z) \,. \tag{37}$$

Theorem 7.1. *The false positive rate of a composed approximate predicate is bounded by the sum of the individual false positive rates.*

Theorem 7.2. *The false negative rate of a composed approximate predicate depends on the logical composition of the individual predicates.*

Theorems 7.1 and 7.2 is the *ideal probabilistic model* for k-ary predicates over *oblivious types*. If some subsets of $[T_1] \times \cdots \times [T_k]$ are more likely to have false positives, and therefore some subsets are less likely to have false positives (and likewise for false negatives), then for instance, determining the probability that a particular tuple is a false positive or a true positive may be done with greater probability than given by our model assumptions.

7.2.1 Total orders

Less-than predicates

8 Discrete partial functions

8.1 0-ary functions (constants)

The special case of the 0-ary predicate is a *constant* **true** or **false**. This may be useful in some cases, notably *cipher* value types. To put it simply, if we have a constant function $\mathrm{f} \colon \infty \mapsto \mathcal{X}$, then $\mathrm{f}(1) \coloneqq y$ and $\widetilde{\mathrm{f}}(1)$ is equal to y with probability $1 - \omega$ and some other value with probability ω.

Basically, approximate constants. In *algebraic data types*, the approximate constant is fundamental. We may compose such things to generate other types, e.g., the Boolean type $\{0, 1\}$ is just $\widetilde{\mathrm{f}}(1) + \widetilde{\mathrm{g}}(1)$, yielding an approximate Boolean type.

9 A first-order rate-distorted map

We present a *theoretical* data type, denoted the *entropy map* that have several key properties:

1. Perfect model of the *first-order* random approximate map.
2. Bit-length of in-place encoding is a random variable L that is expected to obtain the theoretical lower-bound.
3. In-place encoding has maximum conditional entropy (when conditioning on a few parameters like expected false positive rate).

Note that we know a priori what the distribution of the random number of trials N needed to produce a result that satisfies the rate-distortion constraints where a parameterized Goulumb code is expected to compress the representation. However, this representation is not *in-place* and is thus only useful as a *serialization* format.

We consider sets in which the universe of elements is given by the *countably infinite* set of all bit strings.

Definition 9.1. The countably infinite set of all bit strings is denoted by

$$\{0,1\}^* = \left\{b \colon b \in \{0,1\}^*\right\}, \tag{38}$$

where $*$ denotes all non-negative integers.

A *finite* subset of $\{0,1\}^*$ is given by the following definition.

Definition 9.2. The *finite set* of all bit strings of length n is denoted by

$$\{0,1\}^n = \{b \colon b \in \{0,1\}^n\} \tag{39}$$

with a cardinality given by

$$\left|\{0,1\}^n\right| = 2^n . \tag{40}$$

A *hash function* is related to countable sets $\{0,1\}^*$ and $\{0,1\}^n$ and is given by the following definition.

Definition 9.3. A *hash function* of type $\{0,1\}^* \mapsto \{0,1\}^n$ maps (hashes) bit strings of arbitrary-length to bit strings of fixed-length n.

The random oracle is given by the following definition.

Definition 9.4. A random oracle $\mathrm{h} \colon \{0,1\}^* \mapsto \{0,1\}^\infty$ is a theoretical *hash function* whose output is uniformly distributed over its codomain.

The bit length of objects is given by the following definition.

Definition 9.5. The bit length of an object x is denoted by

$$\ell(x) . \tag{41}$$

For example, the bit length of an object $x \in \{0,1\}^n$ is $\ell(x) = n$.

The set of rationals in $[0,1]$ is denoted by $\mathscr{P}$ and the set of power-of-two probabilities is denoted by

$$\mathscr{R} \coloneqq \left\{2^{-k} \in \mathbb{Q} : k \in \mathbb{Z}_{\geq 0}\right\} \subset \mathscr{P} . \tag{42}$$

Suppose we a prefix-free encoder-decoder for values of type $\mathcal{Y}$ denoted respectively by $\texttt{encode}_{\mathcal{Y}} \colon \mathcal{Y} \mapsto \{0,1\}^*$ and $\texttt{decode}_{\mathcal{Y}} \colon \{0,1\}^* \mapsto \mathcal{Y}_\perp$.

If we are given a (possibly infinite) sequence of bits $b_1 b_2 \ldots b_k b_{k+1} \ldots$ and $b_1 b_2 \ldots b_k$ is a (prefix-free) code for some y in $\mathcal{Y}$, then $\texttt{decode}_{\mathcal{Y}}(b_1 b_2 \ldots b_k b_{k+1} \ldots)$ maps to y. Inversely, $\texttt{encode}_{\mathcal{Y}}(y)$ maps to $b_1 b_2 \ldots b_k$.

The singular hash map models a *first-order* random approximate map where the approximation error is due to *rate-distortion*. *Rate-distortion* is a result of two different parameter choices:

1. Distortion due to *insufficient* bits.

2. Distortion due to accepting any false negative rate less than some lower-bound.

3. Distortion due to specifying a false positive rate, which causes elements not in the set to be mapped to be mapped to some uncertain value that is a function of the prefix-free encoder-decoder. If codes assume a uniform distribution, then randomly choose a Y.

In the singular hash map, item 1 is dueo to either a larger *false positive rate* $\varepsilon = 2^{-k}$ by increasing k or restricting the search to a maximum finite number trials (and choosing the trial that resulted in the minimum loss according to some distance function).

In either case, the end result is an expected smaller encoding length, but in the first case we are directly motivated by a rate limit in the form of an upper-bound on the bit length and in the second case we are satisfied with greater approximation error but impose no upper-bound.

In the metric space $(\mathcal{X}, \mathrm{d})$, if d is defined as $\mathrm{d}(a, a) = 1$ and otherwise $\mathrm{d}(a, b) = 0$, $a \neq b$, we get the

The probability of success given a rate-distortion ω on positives and ε on negatives is given by

$$\mathrm{p}(\varepsilon, \omega) = \binom{m}{(1-\omega)m} \varepsilon^{m(1-\omega)} (1-\varepsilon)^{m\omega} . \tag{43}$$

Each trial is geometrically distributed, $\mathrm{T} \sim \mathrm{Geo}(\mathrm{p}(\varepsilon, \omega))$.

The expected number of trials is $\mathrm{E}[\mathrm{T}] = \frac{1}{\mathrm{p}(\varepsilon,\omega)}$. The expected code size is $\mathrm{E}[\log_2 \mathrm{T}]$, which is non-linear and problematic but may be approximated with

$$-\mathrm{H}_b(\omega) - \omega \log_2(1-\varepsilon) - (1-\omega) \log_2 \varepsilon \tag{44}$$

bits per element where $\mathrm{H}_b(p) \coloneqq (1-p)\log_2(1-p) + p \log_2 p$ is the binary entropy function.

This result is not the result of computing $\mathrm{E}[\log_2 \mathrm{T}]$, nor the expectation of its Taylor approximation. It is simply $\log_2 \mathrm{E}[\mathrm{T}]$.

If ω and ε are near 0, then this simplifies to $\omega \log_2 \omega -$

If we let $\omega \coloneqq 0$, the above simplifies to $\log_2 \varepsilon$, which is the lower-bound for *first-order* random positive approximate sets over countably infinite universes with a false positive rate ε.

NB —

Talk about NB distribution.

If we are satisfied with false negatives, we are no longer requiring all m to succeed.

Negative binomial: given n trials, what is the probability k succeeds? If $k = m$, we are back to the previous solution with $\omega = 0$.

The expected false negative rate is just not due to one trial, but an expectation

$$\sum_{k=0}^{m} \mathrm{P}[\mathrm{T} = k] \tag{45}$$

If T realizes ω that means after n trials, the best was ω.

—

9.1 Maybe monad

Suppose the encoder-decoder has the pair $\texttt{encode}_{\mathcal{X}} \colon \mathcal{X} \mapsto \mathcal{P}(\{0,1\}^*)$ and $\texttt{decode}_{\mathcal{X}} \colon \{0,1\}^* \rightharpoonup \mathcal{X}$.

We consider a specialization of the algorithm that, given a function $\mathrm{f} \colon \mathcal{X} \mapsto \mathcal{Y}$ generates rate-distorted maps $\widetilde{\mathrm{f}}_{\varepsilon}^{\omega} \colon \mathcal{X} \mapsto \mathcal{Y}_{\perp}$. The encoder-decoder only applies to elements of $\mathcal{X}$ which test as positive, at which point the elements are mapped to values as previously.

Figure 1: *Singular Hash Map*

In other words, and assuming no rate-distortion, it maps values in the domain of definition of $\mathrm{f}\restriction_{\mathcal{A}}$ to *nothing* with probability ω and otherwise maps it correctly and it maps values not in its domain of definition to *nothing* with probability ε and value y with probability

$$\mathrm{p_Y}(y) \coloneqq \sum_{z \in \mathtt{encode}_{\mathcal{Y}}(y)} 2^{-\ell(z)} \,. \tag{46}$$

The function `generate_hash_seeds`: $(\mathcal{X} \mapsto \mathcal{Y})_\perp \times \mathcal{P}(\mathcal{X}) \times \mathscr{R} \mapsto \mathcal{P}(\{0,1\}^* \times \mathscr{P})$ is defined as

$$\mathtt{generate_hash_seeds}(\mathrm{f}, \mathcal{A}, \varepsilon) \coloneqq \mathrm{T} \tag{47}$$

where

$$\begin{aligned} m_a &\coloneqq |\mathcal{A}| \,, \\ k &\coloneqq -\log_2 \varepsilon \,, \\ \mathrm{h}(x \mid n, k) &\coloneqq \mathrm{tr}(\mathrm{h}(x' +\!\!+ n'), k) \,, \\ \mathtt{match}(x \mid n, k) &\coloneqq \mathrm{h}(x \mid n, k) \,, \\ \mathrm{H} &\coloneqq \left\{ \mathtt{match}(x \mid n, k) \in \{0,1\}^k : x \in \mathcal{X} \right\} \,. \end{aligned} \tag{48}$$

Input: $\mathcal{A}$ is a subset of a universal set $\mathcal{X}$. ε is the false positive rate. t is the maximum number of trials.
Output: $\mathcal{A}^\varepsilon$, a rate-distorted approximate set of $\mathcal{A}$, .
function `construct_singular_hash_map`($\mathcal{A}$, ε, t)

```
A_{0,1}* ≔ {encode_X(a) ∈ {0,1}* : a ∈ A}
for n ← 0 to t do
    for j ← 1 to 2^n do
        found ← true
        b_n ← a bit string randomly drawn from {0,1}^n without replacement
        h_k ← nothing
        for x ∈ A_{0,1}* do
            if h_k = nothing then
                h_k ← tr(h(x ++ b_n), k)
            else if h_k ≠ tr(h(x ++ b_n), k) then
                found ← false
        if found then
            return ⟨h_k, b_n⟩
```

Algorithm 1: Implementation of `construct_singular_hash_map` over a universal set $\mathcal{X}$

In Algorithm 1 on Algorithm 1, the value v_i has a *self-terminating* encoding. Thus, we need only check that the bit string that encodes v_i is equal to a hash of the key (concatenated with another 1 and another bit string b).

The space required for the *singular hash map* found by Algorithm 1 is of the order of the length n of the bit string b. Therefore, for space efficiency, the algorithm exhaustively searches for a bit string in the order of increasing size n.

Input: $\mathcal{A}$ is a subset of a universal set $\mathcal{X}$. ε is the false positive rate. t is the maximum number of trials.
Output: $\mathcal{A}^{\varepsilon}$, a rate-distorted approximate set of $\mathcal{A}$, .
function `construct_singular_hash_map`($\mathcal{A}$, ε, t)
    $\mathcal{A}_{\{0,1\}^*} \coloneqq \{\mathtt{encode}_{\mathcal{X}}(a) \in \{0,1\}^* : a \in \mathcal{A}\}$
    for $i \leftarrow 0$ **to** t **do**
        found $\leftarrow$ **true**
        $b \leftarrow$ a bit string randomly drawn from $\{0,1\}^{\leq t}$ without replacement
        $h_k \leftarrow$ nothing
        for $x \in \mathcal{A}_{\{0,1\}^*}$ **do**
            if $h_k =$ nothing **then**
                $h_k \leftarrow \mathrm{tr}(\mathrm{h}(x + b), k)$
            else if $h_k \neq \mathrm{tr}(\mathrm{h}(x + b), k)$ **then**
                found $\leftarrow$ **false**
        if found **then**
            return $\langle h_k, b_n \rangle$

Algorithm 2: Implementation of `construct_singular_hash_map` over a universal set $\mathcal{X}$

Input: $\mathcal{A}$ is a subset of a universal set $\mathcal{X}$. ε is the false positive rate. t is the maximum number of trials.
Output: $\mathcal{A}^{\varepsilon}$, a rate-distorted approximate set of $\mathcal{A}$, .
function `construct_singular_hash_map`($\mathcal{A}$, ε, t)
    $\mathcal{A}_{\{0,1\}^*} \coloneqq \{\mathtt{encode}_{\mathcal{X}}(a) \in \{0,1\}^* : a \in \mathcal{A}\}$
    for $n \leftarrow 0$ **to** t **do**
        $\mathrm{H} \leftarrow \varnothing$
        for $b \in \mathcal{A}_{\{0,1\}^*}$ **do**
            $\mathrm{H} \leftarrow \mathrm{H} \cup \{\mathrm{tr}(\mathrm{h}(b + n'), k)\}$

Algorithm 3: Implementation of `construct_singular_hash_map` over a universal set $\mathcal{X}$

9.2 Theoretical analysis

The *singular hash set* proves that the keys in $\widetilde{\mathrm{f}}$ are an *approximate set* of the keys in f.

Theorem 9.6. *The probability that a particular bit string* $b \in \{0,1\}^*$ *successfully codes the* singular hash map *is given by*

$$p(m, \varepsilon, \mu) = \frac{\varepsilon^{m-1}}{2^{m\mu}} \,. \tag{49}$$

Proof. In algorithm 1, for a particular bit string b_n, the joint probability that every key in f collides and for each key k in f, the concatenation of 1 and k has a hash in which the first $\ell(v)$ bits collide with mapped value v is given by the following reasoning.

Suppose we have a set $\mathrm{f} \coloneqq \{(k_1, v_1), \ldots, (k_m, v_m)\}$ and k_1 hashes to $y = \mathrm{h}(k_1) \mod t$, where $\mathrm{h}\colon \{0,1\}^* \mapsto \{0,1\}^N$ is a random hash function that uniformly distributes over its domain of 2^N possibilities and $N < \max \ell(v_1), \ldots, \ell(v_m)$.

The probability that $\mathrm{h}(1 + k_1) \mod \ell(v_1) = v_1$ is given by

$$\frac{1}{2^{\ell(v_1)}} \,. \tag{50}$$

Since y is a particular element in $\{0,1\}^k$, the probability that $k_j, j \neq 1$, hashes to y is given by

$$\frac{1}{2^k} \,. \tag{51}$$

The probability that $1 + k_j, j \neq 1$ hashes to v_j is given by

$$\frac{1}{2^{\ell(v_j)}} \,. \tag{52}$$

Since h is a random hash function, every one of these probabilities are independent, and thus the joint probability is just the product given by

$$p = \frac{1}{2^{k(m-1)}} \prod_{j=1}^{m} \frac{1}{2^{\ell(v_j)}} \tag{53}$$

$$= \varepsilon^{m-1} \frac{1}{2^{\sum_{j=1}^{m} \ell(v_j)}} \,. \tag{54}$$

The average bit length per key is given by

$$\mu = \frac{1}{m} \sum_{j=1}^{m} \ell(v_j) \tag{55}$$

and thus the probability p may be parameterized as

$$p = \frac{\varepsilon^{m-1}}{2^{m\mu}} \,. \tag{56}$$

□

Definition 9.7. In the singular hash map construction algorithm, the n-th trial uniquely maps to a bit string of length $m = \lfloor \log_2 n \rfloor$, establishing a correspondence between trial number and bit string length.

Theorem 9.8. *Given* false positive *and* false negative *domain error rates given by ε and ω respectively, and a mean bit length μ of encodings of elements in the image of $\widetilde{\mathrm{f}}$, the* expected *bit length of the Singular Hash Map asymptotically obtains a space complexity with a lower-bound given by*

$$-(1-\omega)\log_2\varepsilon + (1-\omega)\mu \text{ bits/element}\,, \tag{57}$$

which occurs when the encodings have uniformly distributed bit lengths, and an upper-bound given by

$$-(1-\omega)\log_2\varepsilon + \mu \text{ bits/element}\,, \tag{58}$$

which occurs when the encodings have degenerate bit lengths. The greater the entropy of the bit lengths, the smaller the expected bit length given a mean of μ.

Proof. The space required for the singular hash set found by Algorithm 1 is of the order of the length n of the bit string b_n. Therefore, for space efficiency, the algorithm exhaustively searches for a bit string in the order of increasing size n.

We are interested in the first case when a success occurs, which is a geometric distribution with probability of success p as given by

$$\mathrm{Q} \sim \mathrm{Geo}\left(p = \frac{\varepsilon^{m-1}}{2^{m\mu}}\right)\,. \tag{59}$$

The expected number of trials for the geometric distribution is given by

$$\mathrm{E}[\mathrm{Q}] = \frac{2^{m\mu}}{\varepsilon^{m-1}}\,. \tag{60}$$

By Definition 9.7, the n-th trial uniquely maps to a bit string of length $m = \lfloor \log_2 n \rfloor$. Thus, the expected bit length is given approximately by

$$\mathrm{E}[\log_2 \mathrm{Q}] = \log_2 \frac{2^{m\mu}}{\varepsilon^{m-1}} \text{ bits}\,. \tag{61}$$

Dividing by m and simplifying results in

$$-\frac{m-1}{m}\log_2\varepsilon + \mu \text{ bits/element}\,. \tag{62}$$

Asymptotically, as $m \to \infty$, the bits per element goes to

$$-\log_2\varepsilon + \mu\,. \tag{63}$$

□

The smaller the bit lengths generated by the encoder, the smaller the *Singular Hash Map*. However, optimal encoders *reveal* information about the values in the *Singular Hash Map*, which goes against the principle of the *obvlivious function*. For instance, if each input x mapped to a list of numbers, then an optimal encoder will generate codes that minimize the length of the list, say a Huffman code. However, by analyzing the Huffman codes produced by the encoder, we may infer the distribution of the lists. Thus, for the sake of maintaining an *oblivious* state, the encoder should generate codes as though the values were *uniformly* distributed.

9.3 Entropy

10 Conclusion

We have presented a comprehensive algebraic framework for random approximate maps that unifies the analysis of probabilistic data structures. Our key contributions include formal definitions of approximation orders, composition rules for error propagation, and the singular hash map construction that achieves information-theoretic optimality.

10.1 Summary of Results

Our framework establishes several fundamental results:

1. **Algebraic Structure**: Approximate maps form a compositional algebra where error rates combine predictably through operations. The false positive rate of composed maps is bounded by the sum of individual rates, while false negative behavior depends on the logical structure of composition.

2. **Optimality Bounds**: We proved that first-order approximate maps require at least $-\log_2 \epsilon + \mu$ bits per element asymptotically, where ϵ is the false positive rate and μ is the mean encoding length. The singular hash map construction achieves this bound.

3. **Hierarchy of Approximations**: Higher-order approximate maps arise naturally from compositions, with each order capturing increasingly complex error patterns. This hierarchy provides a systematic way to analyze compound operations.

4. **Practical Implications**: Our framework explains the success of existing probabilistic data structures and guides the design of new ones with predictable error characteristics.

10.2 Limitations and Open Questions

Several important questions remain for future investigation:

1. **Dynamic Operations**: Our current framework focuses on static structures. Extending to dynamic settings with insertions, deletions, and updates while maintaining error bounds is an open challenge.

2. **Adaptive Approximations**: Can we design structures that adaptively adjust their approximation quality based on query patterns or available resources?

3. **Distributed Settings**: How do approximation errors compound in distributed systems with network delays and failures?

4. **Lower Bounds for Higher Orders**: While we established bounds for first-order approximations, tight bounds for higher-order compositions remain unknown.

5. **Optimal Encodings**: The singular hash map assumes uniform distribution for maximum entropy. Characterizing optimal encodings for non-uniform distributions is an important direction.

10.3 Future Directions

This work opens several research avenues:

10.3.1 Theoretical Extensions

- Developing approximation theory for more complex algebraic structures (groups, rings, fields)
- Characterizing the class of functions preserving approximation quality
- Establishing connections to other approximation frameworks (PAC learning, property testing)

10.3.2 Practical Applications

- Implementing the singular hash map and evaluating its performance against existing structures
- Applying the framework to design domain-specific probabilistic data structures
- Developing compilers that automatically introduce approximations for space optimization

10.3.3 Cross-Domain Impact

- Machine learning: Using approximate maps for efficient embedding lookups
- Databases: Query optimization with approximation-aware cost models
- Security: Formal verification of probabilistic cryptographic protocols
- Networks: Approximate routing tables and flow monitoring

10.4 Broader Impact

The increasing scale of data processing demands fundamental rethinking of exactness requirements. Our framework provides principled foundations for this paradigm shift, enabling engineers to make informed tradeoffs between space, time, and accuracy. As computing systems grow more complex, compositional reasoning about approximations becomes essential for managing system-wide error budgets.

The algebraic approach we advocate treats approximation as a first-class concern rather than an implementation detail. This perspective shift can influence how we teach algorithms, design programming languages, and architect systems. By making approximation errors explicit and predictable, we enable more reliable and efficient computing at scale.

10.5 Closing Remarks

Random approximate maps represent a fundamental abstraction for space-efficient computing. By establishing their algebraic foundations, we provide both theoretical insights and practical tools for the broader computer science community. We hope this framework stimulates further research into the rich interplay between approximation, randomization, and computation.

The code and supplementary materials for this paper are available at [repository URL].

References

[1] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. *Journal of Computer and System Sciences*, 58(1):137–147, 1996.

[2] Michael A Bender, Martin Farach-Colton, Rob Johnson, Russell Kraner, Bradley C Kuszmaul, Dzejla Medjedovic, Pablo Montes, Pradeep Shetty, Richard P Spillane, and Erez Zadok. Don't thrash: How to cache your hash on flash. In *Proceedings of the VLDB Endowment*, volume 5, pages 1627–1637, 2012.

[3] Richard Bird and Oege de Moor. *Algebra of programming*. Prentice Hall, 1997.

[4] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[5] Oscar Boykin, Sam Ritchie, Ian O'Connell, and Jimmy Lin. Summingbird: A framework for integrating batch and online mapreduce computations. In *Proceedings of the VLDB Endowment*, volume 7, pages 1441–1451, 2013.

[6] Andrei Z Broder. On the resemblance and containment of documents. In *Compression and Complexity of Sequences*, pages 21–29. IEEE, 1997.

[7] Larry Carter, Robert Floyd, John Gill, George Markowsky, and Mark Wegman. Exact and approximate membership testers. *Proceedings of STOC*, pages 59–65, 1978.

[8] David Cash, Stanislaw Jarecki, Charanjit Jutla, Hugo Krawczyk, Marcel-Cătălin Roşu, and Michael Steiner. Dynamic searchable encryption in very-large databases: data structures and implementation. In *NDSS*, volume 14, pages 853–872, 2014.

[9] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. In *International Colloquium on Automata, Languages, and Programming*, pages 693–703. Springer, 2002.

[10] Moses S Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of STOC*, pages 380–388. ACM, 2002.

[11] Graham Cormode and S Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.

[12] Thomas M Cover and Joy A Thomas. *Elements of information theory*. John Wiley & Sons, 2nd edition, 2006.

[13] Changyu Dong, Liqun Chen, and Zikai Wen. When private set intersection meets big data: an efficient and scalable protocol. In *Proceedings of CCS*, pages 789–800. ACM, 2013.

[14] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography Conference*, pages 265–284. Springer, 2006.

[15] Bin Fan, Dave G Andersen, Michael Kaminsky, and Michael D Mitzenmacher. Cuckoo filter: Practically better than bloom. In *Proceedings of CoNEXT*, pages 75–88, 2014.

[16] Li Fan, Pei Cao, Jussara Almeida, and Andrei Z Broder. Summary cache: A scalable wide-area web cache sharing protocol. In *IEEE/ACM Transactions on Networking*, volume 8, pages 281–293. IEEE, 2000.

[17] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm. In *Conference on Analysis of Algorithms*, pages 127–146, 2007.

[18] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Similarity search in high dimensions via hashing. In *Proceedings of VLDB*, volume 99, pages 518–529, 1999.

[19] Joseph A Goguen, James W Thatcher, Eric G Wagner, and Jesse B Wright. Initial algebra semantics and continuous algebras. *Journal of the ACM*, 24(1):68–95, 1977.

[20] Jie Han and Michael Orshansky. Approximate computing: An emerging paradigm for energy-efficient design. In *European Test Symposium*, pages 1–6. IEEE, 2013.

[21] Joseph M Hellerstein, Peter J Haas, and Helen J Wang. Online aggregation. In *Proceedings of SIGMOD*, pages 171–182. ACM, 1997.

[22] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of STOC*, pages 604–613. ACM, 1998.

[23] William B Johnson and Joram Lindenstrauss. Extensions of lipschitz mappings into a hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.

[24] Michael Mitzenmacher and Eli Upfal. *Probability and computing: Randomized algorithms and probabilistic analysis*. Cambridge University Press, 2005.

[25] Rajeev Motwani and Prabhakar Raghavan. *Randomized algorithms*. Cambridge University Press, 1995.

[26] Anna Pagh, Rasmus Pagh, and S Srinivasa Rao. An optimal bloom filter replacement. In *Proceedings of SODA*, pages 823–829. SIAM, 2005.

[27] Claude E Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 4:142–163, 1959.

[28] Alexander Towell. The approximate relation abstract data type. *Technical Report*, 2018. Manuscript.

[29] Alexander Towell. The approximate set abstract data type. *Technical Report*, 2018. Manuscript.